

JAVA DOCS

Oracle Java Documentation — Reference Digest

Language Specification Essentials, Core API Packages & Versioning

Java Agentic AI — Knowledge Base Reference

1. What the Official Documentation Covers

Oracle publishes several distinct reference sources that serve different purposes: the **Java Language Specification (JLS)** defines the exact syntax and semantics of the language itself; the **Java Virtual Machine Specification (JVMS)** defines bytecode and runtime behavior; and the **Java SE API documentation (Javadoc)** catalogs every public class, interface, and method in the standard library. This digest summarizes the practically important parts of each for day-to-day development.

2. Core Package Overview

Package	Purpose
java.lang	Fundamental classes: Object, String, Math, wrapper types, Thread — auto-imported everywhere
java.util	Collections framework, Optional, Scanner, Random, Date/Calendar (legacy)
java.io	Stream-based I/O: InputStream/OutputStream, Reader/Writer, File
java.nio	Non-blocking, buffer-based I/O; java.nio.file (Path, Files) for modern file operations
java.time	Modern date/time API: LocalDate, LocalDateTime, Duration, Period, ZonedDateTime
java.util.concurrent	Executors, concurrent collections, atomics, synchronizers (CountDownLatch, Semaphore)
java.net	Networking: URL, Socket, HttpClient (java.net.http, Java 11+)
java.util.stream	Stream API for functional-style bulk data operations
java.util.function	Functional interfaces: Function, Predicate, Supplier, Consumer
java.sql	JDBC — database connectivity (Connection, Statement, ResultSet)

3. The Object Class — Every Class's Root

Every class implicitly extends java.lang.Object unless it explicitly extends something else. Object defines the methods every Java object inherently has:

Method	Purpose
equals(Object)	Logical equality comparison — default is reference equality (==)
hashCode()	Integer hash used by hash-based collections — must be consistent with equals()
toString()	Human-readable string representation — default is ClassName@hashCodeHex
getClass()	Returns the runtime Class object — the basis for reflection
clone()	Creates a field-for-field copy — requires implementing Cloneable and is generally considered a flawed
wait()/notify()/notifyAll()	Low-level thread coordination primitives tied to the object's monitor
finalize() (deprecated)	Formerly called before GC reclaims an object — deprecated for removal, unreliable timing, use try-v

4. String — Detailed API Reference

```
"hello".length();           // 5
"hello".charAt(1);          // 'e'
"hello".indexOf('l');       // 2
"hello".substring(1, 3);    // "el"
"Hello".equalsIgnoreCase("hello"); // true
" hi ".strip();             // "hi" (Unicode-aware, Java 11+)
"a,b,c".split(",");         // ["a", "b", "c"]
String.join("-", "a", "b", "c"); // "a-b-c"
"abc".repeat(3);            // "abcbcabcb" (Java 11+)
"Hello %s, you are %d".formatted("Sam", 30); // instance formatted() method, Java 15+
"line1\nline2".lines().count(); // 2 (Java 11+ Stream<String> of lines)
```

5. Collections API — Key Interfaces at a Glance

Full detail lives in [Collections_Framework.pdf](#). From the official API perspective, the interface hierarchy is: `Iterable` -> `Collection` -> `{List, Set, Queue}`, with `Map` as a separate root. Every concrete implementation class (`ArrayList`, `HashMap`, etc.) documents its Big-O complexity guarantees directly in its class-level Javadoc — always worth checking before assuming performance characteristics.

6. java.time — The Modern Date/Time API

```
LocalDate.now();
LocalDate.of(2026, Month.JULY, 16);
LocalDateTime.now().plusHours(5);
Duration.between(start, end).toMinutes();
Period.between(startDate, endDate).getDays();
DateTimeFormatter.ISO_LOCAL_DATE;
Instant.now(); // machine-readable UTC timestamp
ZoneId.of("Asia/Kolkata");
```

Class	Represents
<code>LocalDate</code>	Date without time or time zone (2026-07-16)
<code>LocalTime</code>	Time without date or time zone (14:30:00)
<code>LocalDateTime</code>	Date + time, no time zone
<code>ZonedDateTime</code>	Date + time + explicit time zone
<code>Instant</code>	A single instantaneous point on the timeline, UTC-based — ideal for timestamps/logging
<code>Duration</code>	A time-based amount (hours, minutes, seconds)
<code>Period</code>	A date-based amount (years, months, days)

7. java.util.concurrent — Executor Framework Reference

```
ExecutorService pool = Executors.newFixedThreadPool(4);
Future<Integer> f = pool.submit(() -> compute());
```

```
pool.shutdown();
pool.awaitTermination(5, TimeUnit.SECONDS);

CompletableFuture.supplyAsync(() -> fetchData())
    .thenApply(data -> process(data))
    .thenAccept(result -> save(result));
```

8. Understanding Semantic Versioning in the JDK

Since Java 10, the JDK follows a strict time-based release model: a new feature release every six months, with every third release (roughly every 2 years, e.g. 11, 17, 21) designated LTS (Long-Term Support). Version numbers follow \$MAJOR.\$MINOR.\$SECURITY (e.g. 21.0.2), where the feature release number (21) is what most tooling/compatibility discussions reference.

Java version	Release	LTS?
8	March 2014	Yes
11	September 2018	Yes
17	September 2021	Yes
21	September 2023	Yes
25 (upcoming per the 6-month cadence)	Expected September 2025	Yes

9. Reading Javadoc Effectively

- Class-level Javadoc usually documents thread-safety guarantees and complexity characteristics — always check before assuming a collection or utility is safe to share across threads.
- '@since' tags tell you the minimum Java version required — critical when a project targets an older runtime.
- '@deprecated' tags explain WHY something is deprecated and what to use instead — read it rather than ignoring the warning.
- Method-level '@throws' entries document both checked and (commonly) important unchecked exceptions a method can raise.

10. Quick Interview Recap

- What's the difference between the JLS and the JVMS? The JLS specifies the Java language itself (syntax, semantics); the JVMS specifies the bytecode format and runtime behavior the JVM must implement, independent of any particular source language.
- Which Java releases are LTS? Every third six-month release since Java 10's cadence began — notably 11, 17, and 21 — receiving extended vendor support.
- Why is Object.finalize() deprecated? Its invocation timing (or whether it runs at all before JVM shutdown) is unreliable, it can resurrect otherwise-dead objects, and it complicates GC — try-with-resources and java.lang.ref.Cleaner are the modern replacements for deterministic cleanup.

- What does Instant represent versus LocalDateTime? Instant is an absolute, time-zone-independent point on the UTC timeline (ideal for timestamps); LocalDateTime is a date+time with no attached time zone at all, useful for representing a 'wall clock' time without absolute meaning.